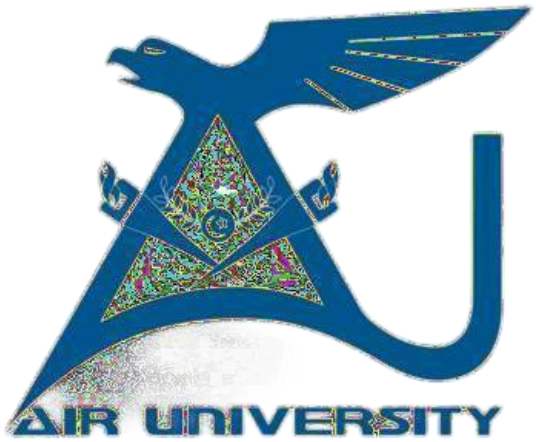

ReqGit - Version Control and Team Collaboration Platform for SRS Documents

Group Member Name:

Abdullah Saleem (233714)

Muhammad Zain Tariq

Subject:

Full Stack Web Development

Instructor:

Sayed burhanuddin

Table of Contents

1. Project Title

ReqGit - Version Control and Team Collaboration Platform for SRS Documents

2. Introduction

2.1. Problem Statement

During the Software Development Life Cycle (SDLC), requirement documents undergo continuous revisions. Traditional methods of sharing these documents (e.g., email threads, generic cloud drives) lead to:

- Loss of previous baseline requirements when files are overwritten.
- Confusion over which document is the "latest" or "approved" version.
- Lack of granular access control (Admin, Editor, Viewer) within specific projects**.**

2.2. Proposal Solution

ReqGit is a specialized document management system that introduces strict version control principles to SRS documents. When a document is updated, the system does not overwrite the old file; instead, it creates a new version node, maintaining a complete and restorable history of the document's evolution. It also provides isolated workspaces for teams to collaborate securely.

3. System Architecture & Technology Stack

ReqGit is built on a robust, decoupled Client-Server architecture to ensure high performance and scalability.

3.1. Frontend

- **Framework:** React.js (Single Page Application for seamless navigation without page reloads).
- **Styling:** Tailwind CSS (For a responsive, professional, and consistent SaaS-like user interface).
- **Routing:** React Router DOM (For secure and dynamic page navigation).

3.2. Backend

- **Language:** Core PHP (Object-Oriented Programming with PDO for secure database interactions).
- **Architecture:** RESTful APIs delivering JSON responses to the frontend.
- **Security:** Prepared statements to prevent SQL injection, and hashed passwords for user security.

3.3. Database

- **System:** MySQL (Relational Database Management System).
- **Design:** A fully normalized database utilizing foreign keys with ON DELETE CASCADE constraints to maintain strict referential integrity between users, workspaces, and document logs.

4. Core Modules & Implementation Details

4.1. Public Landing Page & Authentication

- **Landing Page:** A fully responsive public-facing page featuring smooth scrolling navigation (Features, Solutions, Pricing, Documentation) and dynamic interactive states.
- **Authentication System:** Secure Sign-Up and Log-In interfaces with input validation. Includes a complete "Forgot Password" workflow interface.

4.2. User Dashboard & Workspace Management

- **Dashboard Hub:** Provides a personalized overview welcoming the user, displaying their assigned workspaces, and showing a real-time activity feed of recent changes across their teams.
- **Workspace Creation:** Users can instantiate new "Projects" or "Global Workspaces."
- **Role-Based Access Control (RBAC):** Workspace administrators can dynamically invite members via email and assign specific roles (Admin, Editor, Viewer), directly updating the workspace_members junction table in the database.

4.3. Document Repository

- **File Management:** A dedicated workspace view where members can upload .pdf, .docx, and .md files up to 50MB using a drag-and-drop modal.
- **Metadata Tracking:** The system tracks the document status (e.g., Draft, Approved, Deprecated), the uploader's identity, and the exact timestamp of the upload.

4.4. Version Control Engine (The "Git" Feature)

- **Historical Auditing:** The core feature of ReqGit. Users can view a vertical timeline of a document's history.
- **Iteration Tracking:** Each update logs a specific version label (e.g., v1.0, v1.1), a summary of changes (commit message), and the author.
- **Restoration:** Users can securely download past baseline versions of an SRS document if project requirements are rolled back.

5. Database Schema Overview

The system relies on a highly structured MySQL database. Key tables include:

1. **users**: Stores authentication credentials, profile pictures, and bios.
2. **workspaces**: Stores the core data for team environments.
3. **workspace_members**: A pivot table managing the many-to-many relationship between users and workspaces, enforcing role constraints.
4. **documents**: Stores the high-level metadata for an uploaded SRS file.

5. **document_versions**: The historical ledger linked to documents, storing physical file paths and incremental update logs.

6. Software Quality Assurance (SQA) & Testing

To ensure reliability, the system underwent rigorous testing:

- **UI/UX Testing:** Verified that all Tailwind CSS components are fully responsive across mobile, tablet, and desktop viewports.
- **Functional Testing:** Validated all primary workflows, including user registration, workspace creation, member invitation, and document uploading.
- **Database Integrity Testing:** Ensured that deleting a workspace successfully cascades and removes all associated documents and member records, preventing orphaned data.

7. Conclusion

The ReqGit platform successfully achieves its objective of modernizing how software engineering teams handle requirements documentation. By combining the intuitive UI of a modern web application with the strict historical tracking of version control systems, ReqGit eliminates document confusion, enhances team collaboration, and ensures a single source of truth for all project specifications. The decoupled React and PHP architecture ensures that the platform remains scalable and maintainable for future enhancements.